



MODULE 12

Android UI Development

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Phase 03 | Android Development with Kotlin

Module 12 Roadmap

Seven blocks. By the end: a working, accessible, animated PNC Mobile screen in Compose.



Jetpack Compose Fundamentals

- The @Composable Function
- Recomposition
- remember & mutableStateOf



@Composable Functions

- A Jetpack **composable** is a regular Kotlin function marked with the `@Composable`
- Composables define a piece of user interface using Android's modern declarative UI toolkit
- They return other composables that together will build the app's user interface

Code Bite

```
@Composable
fun BalanceCard(accountName: String,
                balance: Double) {

    Column() {
        Text(
            text = accountName
        )
        Text(
            text = "$%.2f".format(balance)
        )
    }
}
```



Displaying Text

- The Text() function creates a composable chunk of text content
- It has numerous properties that can be set to change how the text is rendered

Code Bite

```
Text(  
    text = "Account balance: ${balance}",  
    modifier = Modifier.padding(all = 8.dp),  
    color = Color.Blue,  
    fontWeight = FontWeight.Bold,  
    textAlign = TextAlign.Center  
)
```



Displaying Buttons

- The Button() function renders a button that can execute some code when it is tapped
- Their primary property is an onClick function, usually provided as a lambda
- It has no property for text to display
 - Instead, you provide nested content for it to display

Code Bite

```
Button(  
    onClick = {  
        /* Handle button click */  
    },  
    modifier = Modifier  
        .padding(16.dp)  
        .fillMaxWidth(),  
    shape = RoundedCornerShape(8.dp),  
    enabled = true,  
    contentPadding = PaddingValues(12.dp)  
) {  
    Text(text = "Click Me")  
}
```



Displaying Images

- You can display an image using the Image composable
- The native Image composable function only supports displaying images that were added to the project as resources
 - Located in the res/drawable folder

Code Bite

```
Image(  
    painter = painterResource(  
        id = R.drawable.my_image_name),  
    contentDescription = "alt text here",  
    modifier = Modifier.size(200.dp)  
)
```



Remote Images

- If your image needs to be downloaded from the internet, you need a third-party library
- **Coil** is a popular choice, which provides an `AsyncImage` composable function

Code Bite

```
implementation("io.coil-kt:coil:2.7.0")

AsyncImage(
    model =
        "https://example.com/images/dog.jpg",
    contentDescription = "alt text here",
    modifier = Modifier.size(200.dp)
)
```



Previews

- You can preview your UI by annotating a composable function with `@Preview`
 - Write a new function with no parameters
 - Add the `@Preview` annotation above the `@Composable` annotation
 - Call your target UI component inside this preview function
 - If your target UI component needs any parameters provide hard-coded mock values

Code Bite

```
@Preview(showBackground = true)
@Composable
fun BalanceCardPreview() {
    BalanceCard(
        accountName = "Everyday Checking",
        balance = 4281.16
    )
}
```



Scaffold

- A **scaffold** is a layout container that provides a framework for a screen
 - Based on Material Design guidelines
- It accepts several parameters as slots for standard screen elements
 - topBar - app bar or toolbar
 - bottomBar - navigation or app bar
 - floatingActionButton - primary action button in the bottom-right corner
 - snackbarHost - manages popup messages
 - content - primary screen content
- It receives an innerPadding parameter from the o/s
 - Apply it to your content to stay away from system bars

Code Bite

```
Scaffold(  
    topBar = { . . . }  
    bottomBar = { . . . }  
) { innerPadding ->  
    MyMainAppComposable(  
        . . .  
        modifier = Modifier  
            .padding(innerPadding)  
    )  
}
```



Recomposition

- Recomposition is the process where Compose re-runs your composable functions to update the UI when underlying data or states change
- Compose intelligently targets and re-executes only the specific components or lambdas that read the changed state, skipping the rest of the UI tree
- Compose starts recomposition when parameters might have changed
 - If a parameter changes again before recomposition finishes, Compose cancels the current run and restarts it with the new input
- If a composable's inputs are stable and have not changed since the last run, Compose completely skips recomposing it
- Keep composables side-effect free
 - Because recomposition can be skipped or repeated, never rely on side-effects inside your UI functions
 - Trigger side-effects via explicit callbacks
- Recomposition is cheap and expected, not a design flaw to work around



remember and mutableStateOf

- Composable functions re-execute during recomposition
- Variables in the functions get re-created every time the function executes
- They have nowhere to store data between executions
- The remember function will cache a value so it can be used on successive renders
- mutableStateOf creates an observable so that Compose can know when composables need to be recomposed

Code Bite

```
@Composable
fun TransferAmountField() {

    var amountState = remember {
        mutableStateOf(0)
    }

}
```



Using mutableStateOf Values

- mutableStateOf returns a MutableState object
- Its **value** property is what the composables need to read and write

Code Bite

```
@Composable
fun TransferAmountField() {

    TextField(
        value = amountState.value,
        onChange = {
            amountState.value = it
        },
        label = { Text("Amount") }
    )

}
```



The by Delegate

- The **by** delegate automates the access of the value from the MutableState object
- Using the **by** delegate allows the composable to maintain the raw property value
- Code can access the raw value directly
- Assigning a new value to the raw variable triggers the setting of the value of the MutableState object
 - And therefore a recomposition

Code Bite

```
@Composable
fun TransferAmountField() {
    var amount by remember {
        mutableStateOf("")
    }

    TextField(
        value = amount,
        onChange = { amount = it },
        label = { Text("Amount") }
    )
}
```



Block 1 - Jetpack Compose Fundamentals

Skill Check ✓

- What does remember actually do, and what would happen to state without it?
- What triggers a recomposition?
- Why is a Composable described as a 'disposable description of UI,' not a persistent object?



Android Views & XML Layouts

- The Traditional View System
- XML Layouts
- ViewBinding
- When Enterprise Teams Still Use Views



XML Views

- Android applications used to be built with XML defining each view
- An XML View is a user interface element declared inside an XML layout file
 - Layout XML files must be saved in the res/layout/ directory
- Every XML view requires specific attributes to compile properly:
 - android:id - assigns a unique id to each view
 - android:layout_width, android:layout_height - sizes the view
 - android:orientation - forces nested views to stack either vertically or horizontally
 - @+id/xxxx - registers unique resource ids for XML elements so you can access them from your code
- They require separate Kotlin files to provide the data and behavior for the UI
 - The compiler creates a binding class from the XML file
 - This binding class lets the Kotlin file interact with
- Many legacy applications still use XML views



XML Layouts & the View System

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <TextView
        android:id="@+id/balanceLabel"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

</LinearLayout>
```



Using the Binding Class

@Composable

```
fun BalanceDisplayCard(balance: Double) {  
    AndroidViewBinding(  
        factory = BalanceDisplayBinding::inflate  
        // assumes the view is called balance_display.xml  
    ) {  
        balanceLabel.text = "Balance ${balance}"  
    }  
}
```



Why This Still Matters



Legacy codebases

Most established enterprise Android apps have years of View-based screens still in production



Incremental migration

Compose can be adopted screen-by-screen via ComposeView, mirroring UIHostingController from Module 6



Some libraries still assume Views

Certain mature third-party SDKs and system integrations remain View-based



Block 2 - Android Views & XML Layouts

Skill Check ✓

- What does ViewBinding generate, and what problem does it solve compared to findViewById?
- What's the Compose equivalent of incrementally adopting SwiftUI inside a UIKit app via UIHostingController?
- Why would an enterprise Android team still be maintaining View-based screens today?



Navigation Components

- NavHost & NavController
- Passing Arguments
- Deep Links



Navigation Compose and Serialization

- Navigation Compose is a Jetpack Navigation library that lets you navigate between different screens and pass data
- It provides a declarative way to handle navigation and features:
 - **Back Stack Management:** Automatically handles system back button gestures, up actions, and clearing history stacks
 - **Argument Passing:** Supports sending data securely and cleanly between different screens.
 - **Animations & Transitions:** Provides built-in support for standard screen transition animations.
 - **Deep Linking:** It can direct users straight to deep-linked destinations inside your app.
 - **ViewModel Scoping:** Scopes ViewModels to individual navigation graphs to share state between related screens



Routes

- Routes are subclasses of a sealed class (or interface)
- They represent a screen that can be navigated to
- Use a data class to model the data that needs to be sent to the composable being displayed
 - Best practice is to send the least amount of data that will suffice
- Use a data object if the route does not need any data sent to the composable being displayed

Code Bite

```
sealed interface MyRoute

@Serializable
data object DashboardRoute: MyRoute

@Serializable
data object AccountListRoute: MyRoute

@Serializable
data class AccountDetailsRoute(
    val accountId: Int
): MyRoute
```



NavController

- The central coordinator that tracks the back stack, moves forward or backward through screens, and executes navigation commands
- It is created using `rememberNavController()`
- Its `navigate()` method will navigate to a new screen
- Its `popBackStack()` method goes back to the previous screen
- Avoid passing NavController to child screens
 - Instead, pass callback functions that will do the desired navigation

Code Bite

```
val navController = rememberNavController()

navController.navigate(AccountListRoute)

navController.popBackStack()
```



NavHost

- A container composable that links your NavController to a navigation graph
- It will display the active screen destination based on the current route
- Its content will be conditionally rendered by the **composable** generic function
 - Only invocations that match the activated route will get rendered

Code Bite

```
NavHost(  
    navController = navController,  
    startDestination = DashboardRoute  
) {  
    composable<DashboardRoute> {  
        DashboardScreen()  
    }  
    composable<AccountRoute> {  
        AccountScreen()  
    }  
}
```



Getting Data From the Route

- A lambda is called when a route is triggered
- Its returned composable is what will ultimately be rendered
- This lambda will be given a `BackStackEntry` for the current route
- It can be cast to the specific route type to get access to the route data

Code Bite

```
composable<ArtistDetails>{ backStackEntry ->
    val route =
        backStackEntry.toRoute<ArtistDetails>()
    val artist =
        repository.getById(route.artistId)

    if (artist != null) {
        ArtistDetails(artist = artist)
    } else {
        NotFound()
    }
}
```



Deep Links

- Deep links in **Jetpack Compose Navigation** connect external links (like web URLs or notification clicks) directly to specific screens inside your app
- The Kotlin serialization plugin is needed
- As are the Navigation Compose and the Serialization JSON libraries
- The first part of configuring deep links is to assign a list of `navDeepLinks` to the `deepLinks` property on the composable route

Code Bite

```
composable<ArtistDetailsRoute>(  
    deepLinks = listOf(  
        navDeepLink<ArtistDetailsRoute>(  
            basePath =  
                "https://events.example.com/artists"  
        )  
    )  
) { backStackEntry ->
```



Configuring Deep Links

- AndroidManifest.xml needs to be configured to provide for deep links
- Intent of making the content viewable and browsable must be specified
- The base URL used in the route also needs to be specified

Code Bite

```
<intent-filter>
  <action
    android:name="android.intent.action.VIEW" />
    <category
      android:name="android.intent.category.DEFAULT" />
    <category
      android:name="android.intent.category.BROWSABLE"
    />
    <data android:scheme="https"
      android:host="events.example.com" />
</intent-filter>
```



Block 3 - Navigation Components

Skill Check ✓

- What are the two core pieces of Navigation Compose, and what does each one do?
- How does a destination receive a typed argument like an account ID?
- What problem does a deep link solve that normal in-app navigation doesn't?



Lists: RecyclerView & LazyColumn

- RecyclerView: Adapter & ViewHolder
- LazyColumn



RecyclerView

- Screens displaying a large number of items can be inefficient
 - It is expensive to create dozens or hundreds of UI elements when only a few can display onscreen at one time
- The RecyclerView was created to manage XML views like this
- It efficiently displays large sets of data with high performance and low memory usage
- It manages a small pool of UI elements
 - As items are scrolled off the screen, they are recycled
 - They get re-bound to a different data object to be scrolled back onscreen



RecyclerView: Adapter & ViewHolder

```
class AccountAdapter(private val accounts: List<Account>) :  
    RecyclerView.Adapter<AccountAdapter.ViewHolder>() {  
  
    class ViewHolder(val binding: ItemAccountBinding) : RecyclerView.ViewHolder(binding.root)  
  
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {  
        val binding = ItemAccountBinding.inflate(LayoutInflater.from(parent.context), parent, false)  
        return ViewHolder(binding)  
    }  
  
    override fun onBindViewHolder(holder: ViewHolder, position: Int) {  
        holder.binding.nameLabel.text = accounts[position].name  
    }  
  
    override fun getItemCount() = accounts.size
```



LazyColumn and LazyRow

- Compose uses LazyColumn or LazyRow for the same purpose
- The framework automatically handles the recycling and composition of elements
 - They only compose and lays out items that are currently visible on the screen
- Unlike typical layouts in Compose that accept standard composable children, they use a Domain-Specific Language (DSL) block via a LazyListScope
 - You populate it using functions like item() for a single component or items() to render a collection



LazyColumn

- The **items** property will drive the collection of UI elements
- Its trailing lambda will transform the collection of data into composables
- The key extractor function is required
 - This allows the LazyColumn to identify which UI element corresponds to each collection object
 - The keys must be unique across the collection

Code Bite

```
LazyColumn(  
    modifier = Modifier  
        .fillMaxSize()  
        .padding(innerPadding)  
) {  
    items(  
        items = accounts,  
        key = { account -> account.id }  
    ) { artist ->  
        ArtistCard(artist = artist)  
    }  
}
```



Block 4 - Lists: RecyclerView & LazyColumn

Skill Check ✓

- Name the three collaborating pieces in the RecyclerView pattern and what each one is responsible for.
- What does the `key = { it.id }` parameter in LazyColumn's `items()` actually do, and why does it matter?
- What single Compose function replaces Adapter, ViewHolder, and manual view recycling entirely?

